



• BUILD DAY • PHASE 3 CAPSTONE

Build it together.

Today we combine everything from Phase 3 — PDF extraction (Day 11), TF-IDF (Day 13), Streamlit UI (Days 4–5) — into one real app. A resume matcher. The same technique behind every ATS system.

PROGRAM Python Internship

LED BY Zlaark

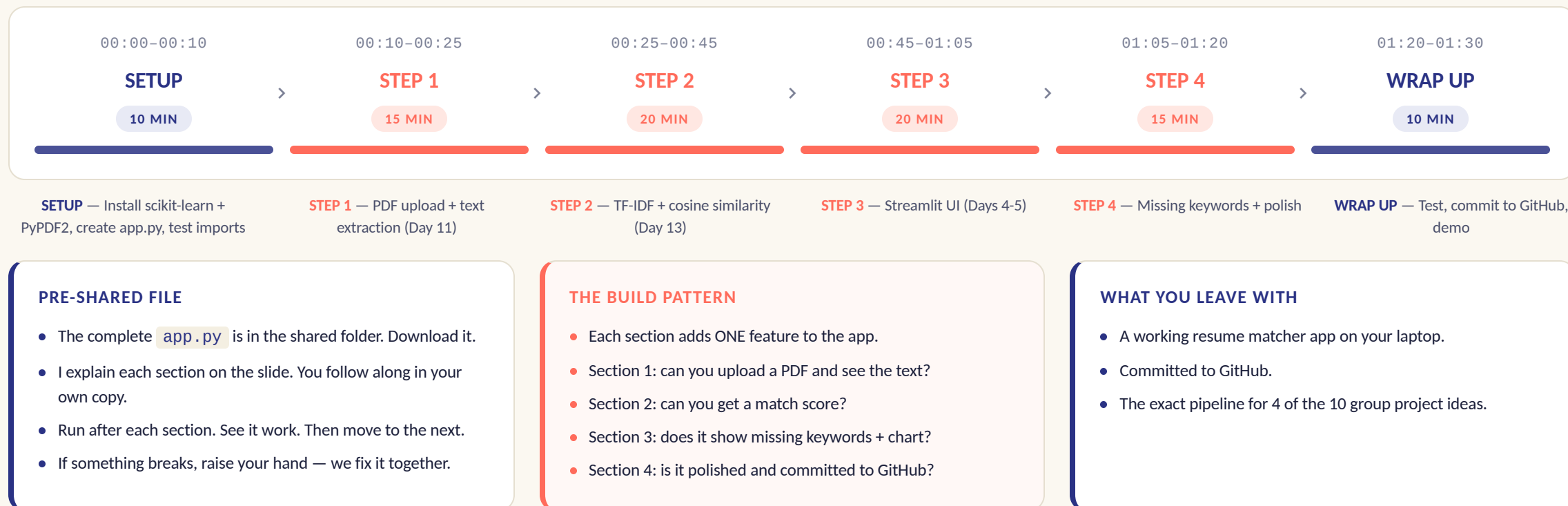
SESSION 14 of 25

TODAY'S BUILD

Resume Matcher App

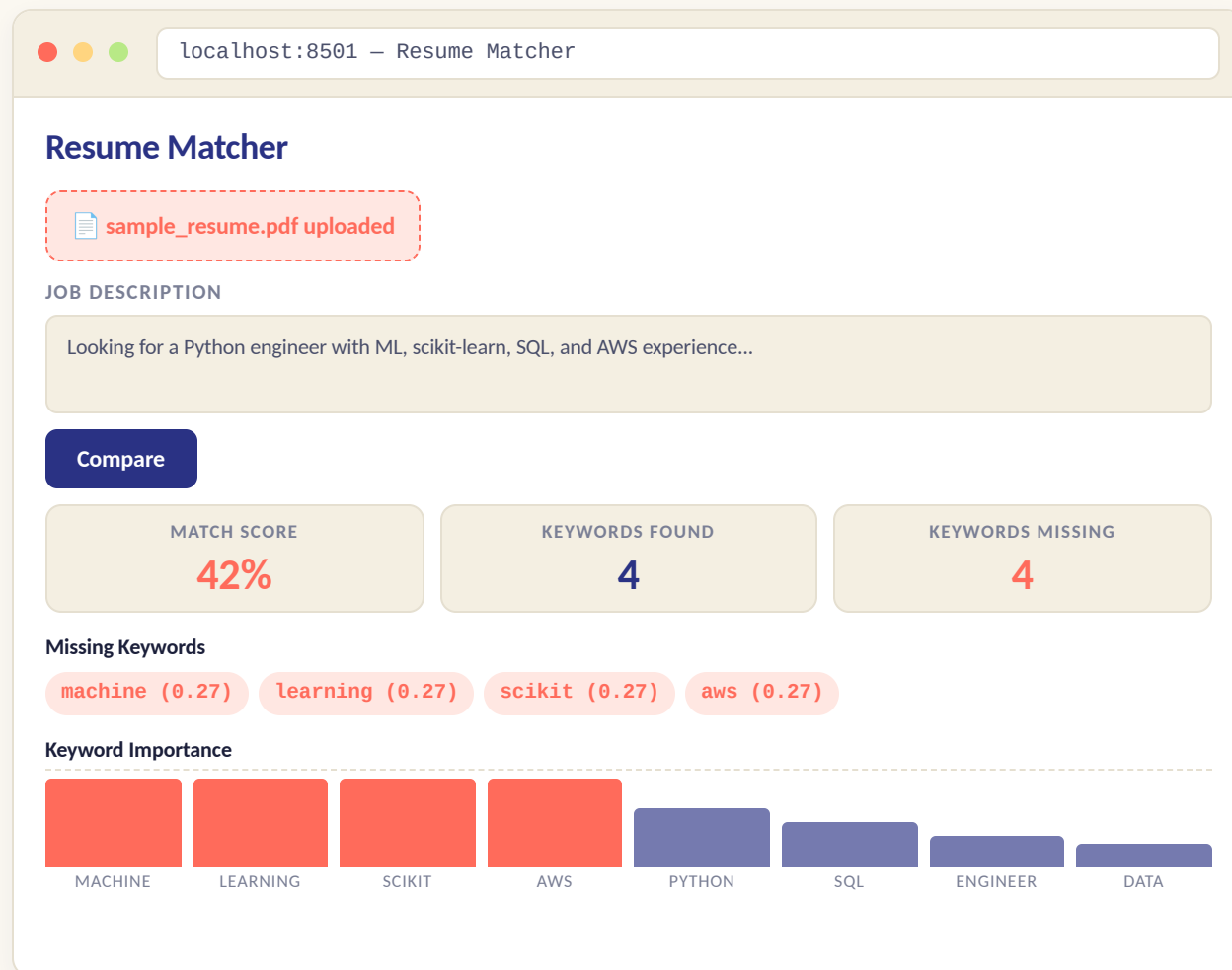
Ten minutes of setup. Eighty minutes of building.

No new concepts today. The complete `app.py` is in the shared folder. Download it, open it in VS Code, and we'll walk through it together section by section. You run after each section to see it work.



A Streamlit app that matches resumes to job descriptions.

Upload a resume PDF. Paste a job description. Get a match score + missing keywords + a chart. The full pipeline in one app.



THE FEATURES

- 1 **PDF upload** — `st.file_uploader` — user uploads their resume as a PDF
- 2 **Text extraction** — PyPDF2 reads the PDF, extracts text, cleans with regex (Day 11)
- 3 **Match score** — TF-IDF + cosine similarity — a percentage from 0% to 100% (Day 13)
- 4 **Missing keywords** — High TF-IDF words in the job description that are NOT in the resume
- 5 **Keyword chart** — `st.bar_chart` showing the importance of each keyword (Day 7)

THE PIPELINE

Upload PDF → Extract text → Clean text → TF-IDF vectorize → Cosine similarity → Score + Missing keywords → Display in Streamlit. 7 steps, 4 days of learning, 1 app.

• STEP 0 • SETUP (10 MIN)

Install the libraries. Download app.py. Test imports.

Before we build, make sure everything is installed. This is the only part where you might need to wait for pip.

```
setup.sh

# 1. Activate your virtual environment
source .venv/bin/activate      # macOS/Linux
# OR
.venv\Scripts\Activate.ps1    # Windows

# 2. Install the libraries we need today
pip install streamlit scikit-learn PyPDF2
# streamlit      - the UI (Days 4-5)
# scikit-learn   - TF-IDF + cosine similarity (Day 13)
# PyPDF2         - PDF text extraction (Day 11)

# 3. Create the app file
mkdir day14
cd day14
touch app.py

# 4. Open in VS Code
code app.py

# 5. Test that imports work - check app.py has these imports:
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from PyPDF2 import PdfReader
import re

st.title("Resume Matcher")
st.write("Imports work! Ready to build.")

# 6. Run it
streamlit run app.py
```

✓ **Virtual environment activated** (.venv visible in prompt)

✓ **app.py downloaded** and open in VS Code

✓ **streamlit run app.py** opens the browser

✓ **pip install succeeded** (no errors)

✓ **All 5 imports work** (no ImportError)

✓ **You see** "Imports work! Ready to build."



IF ANY IMPORT FAILS — raise your hand. Most common issue: forgot to activate .venv, or pip install failed. We fix in 2 minutes.

• BUILD STEP 1 OF 4 • PDF UPLOAD + EXTRACTION (15 MIN)

Upload a PDF. Extract text. Clean it.

Day 11 skills. The user uploads their resume, we read it with PyPDF2, clean the text with regex, and display it.

```
app.py (Step 1)

import streamlit as st
from PyPDF2 import PdfReader
import re

st.title("Resume Matcher")

# --- STEP 1: Upload + Extract ---
uploaded = st.file_uploader(
    "Upload your resume (PDF)",
    type=["pdf"],
    key="resume_upload"
)

if uploaded is not None:
    # Read the PDF
    reader = PdfReader(uploaded)
    raw_text = ""
    for page in reader.pages:
        raw_text += page.extract_text() + "\n"
    # Clean the text (Day 11 regex)
    cleaned = re.sub(r'\S+@\S+', '', raw_text) # remove email
    cleaned = re.sub(r'\+?\d[\d\s-]{8,}\d', '', cleaned) # remove phone
    cleaned = re.sub(r'\s+', ' ', cleaned) # normalize whitespace
    cleaned = cleaned.strip()
```

CHECKPOINT

After running Step 1, you should see:

- ✓ File uploader appears on the page
- ✓ Upload a PDF — text appears in the text area
- ✓ The text is CLEANED (no email, no phone, single spaces)
- ✓ The success message shows character count + page count
- ✓ If you upload a non-PDF, it rejects it

COMMON ISSUES

(1) PDF has no extractable text (scanned PDF). (2) Text is jumbled (multi-column PDF). (3) File uploader returns None if no file uploaded — check `if uploaded is not None:`

🔗 DAY 11 CONNECTION — This is the exact pipeline from Day 11: `PdfReader` → `extract_text()` → `re.sub` cleaning. Today we add TF-IDF on top of it.

• BUILD STEP 2 OF 4 • TF-IDF + MATCH SCORE (20 MIN)

Add the job description. Compute the match score.

Day 13 skills. The user pastes a job description, we vectorize both texts with TF-IDF, compute cosine similarity, and show the score.

<> app.py (add Step 2 below Step 1)

PYTHON · ~30 LINES

```
# --- STEP 2: TF-IDF + Match Score ---
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Job description input
job_desc = st.text_area(
    "Paste the job description",
    height=150,
    key="job_desc_input",
    placeholder="Looking for a Python engineer with..."
)

# Compare button
if st.button("Compare", key="compare_btn", type="primary"):
    if not job_desc.strip():
        st.warning("Please paste a job description first.")
    elif not cleaned:
        st.warning("Please upload a resume first.")
    else:
        # TF-IDF vectorize both texts
        vectorizer = TfidfVectorizer(stop_words='english')
        vectors = vectorizer.fit_transform([cleaned, job_desc])

        # Cosine similarity
        score = cosine_similarity(vectors[0], vectors[1])[0][0]

        # Show the score
        st.subheader("Results")
```

🚩 CHECKPOINT GATE TO STEP 3

After running Step 2, you should see:

- ✓ Job description text area appears
- ✓ Compare button appears
- ✓ Clicking Compare shows a match score (e.g., 42%)
- ✓ A verdict label (Strong / Partial / Weak)
- ✓ A progress bar visualizing the score
- ✓ Error handling: warning if job desc or resume is missing

💡 WHAT THE SCORE MEANS

0-30%: weak match (few shared keywords). 30-50%: partial match (some overlap). 50%+: strong match (good keyword alignment). The threshold depends on your use case.

DAY 13 CONNECTION — This is the exact 5-line pattern from Day 13: TfidfVectorizer → fit_transform → cosine_similarity → [0][0]. Today we wrap it in a UI.

• BUILD STEP 3 OF 4 • MISSING KEYWORDS + CHART (20 MIN)

Show what's missing. Show what matters.

The most valuable part of the app. Find keywords from the job description that are NOT in the resume. Show their importance as a bar chart.

<> app.py (add Step 3 inside the Compare button block)

PYTHON · 47 LINES

```
# --- STEP 3: Missing Keywords + Chart ---
import numpy as np
import pandas as pd

# Get the vocabulary and TF-IDF weights
feature_names = vectorizer.get_feature_names_out()
job_vector = vectors[1].toarray()[0] # job desc weights
resume_vector = vectors[0].toarray()[0] # resume weights

# Find important keywords in job desc
# that have LOW weight in the resume (missing!)
missing = []
found = []
for i, weight in enumerate(job_vector):
    if weight > 0.05: # threshold for "important"
        word = feature_names[i]
        if resume_vector[i] < 0.01: # not in resume
            missing.append({"keyword": word, "importance": weight})
        else:
            found.append({"keyword": word, "importance": weight})

# Sort by importance
missing.sort(key=lambda x: -x["importance"])
found.sort(key=lambda x: -x["importance"])

# Show missing keywords
if missing:
    st.subheader(f"Missing Keywords ({len(missing)})")
    missing_df = pd.DataFrame(missing)
    st.dataframe(missing_df, use_container_width=True, hide_index=True)

# Show found keywords
if found:
    st.subheader(f"Keywords Found ({len(found)})")
    found_df = pd.DataFrame(found)
    st.dataframe(found_df, use_container_width=True, hide_index=True)

# Bar chart of all keyword importance
all_keywords = pd.DataFrame({
    "keyword": [feature_names[i] for i in range(len(job_vector)) if job_vector[i] > 0.05],
    "importance": [job_vector[i] for i in range(len(job_vector)) if job_vector[i] > 0.05]
```

🚩 CHECKPOINT GATE TO STEP 4

After running Step 3, you should see:

- ✓ Missing keywords table appears (words + importance scores)
- ✓ Found keywords table appears
- ✓ A bar chart shows keyword importance
- ✓ The chart is sorted (most important at the top)
- ✓ The dataframes are interactive (sortable columns)

WHY THIS MATTERS

The missing keywords are the **actionable output**. A student sees "machine learning" and "AWS" are missing → they know what to add to their resume. This is exactly what real ATS systems show.

DAY 7 CONNECTION — The bar chart uses `st.bar_chart` from Day 7. The dataframes use `st.dataframe` from Day 6. Everything connects.

Polish the app. Commit to GitHub. You're done.

Add a few finishing touches, test the full flow, then commit to GitHub. The app you built today is the template for 4 of the 10 group project ideas.

✦ POLISH CHECKLIST

5 TOUCHES

1 Add a title + description

```
st.title("Resume Matcher")
st.write("Upload your resume and paste a job description to see how well they match.")
```

2 Add a sidebar with info

```
with st.sidebar: st.write("***How it works**") + st.write("1. Upload resume PDF\n2. Paste job description\n3. Click Compare")
```

3 Handle the "no resume uploaded" state

```
if uploaded is None: st.info("Upload a resume to begin.")
```

4 Add error handling for empty job description

```
if not job_desc.strip(): st.warning("Paste a job description.")
```

5 Add a footer

```
st.divider() + st.write("Built with Python, Streamlit, scikit-learn. The same technique behind real ATS systems.")
```

🔗 COMMIT TO GITHUB

SHELL · git

```
# In your terminal:
cd day14

# Initialize git (if not already)
git init
git remote add origin https://github.com/YOURNAME/day14-resume-matcher.git

# Add a .gitignore
echo "__pycache__/"
.venv/
*.pyc
.DS_Store" > .gitignore

# Commit
git add app.py .gitignore
git commit -m "Resume matcher – PDF + TF-IDF + Streamlit"
git push -u origin main
```

✔ WHAT TO TEST

- ✓ Upload the sample resume PDF.
- ✓ Paste the sample job description.
- ✓ Click Compare.

WHAT YOU BUILT — In 80 minutes, you combined Day 11 (PDF extraction) + Day 13 (TF-IDF) + Days 4-5 (Streamlit UI) + Day 7 (charts) + Day 6 (pandas dataframes) into ONE app. That's 5 days of learning in one file. This is the group project pattern.

You just built an AI-powered app.

In 80 minutes. From scratch. Using 5 days of learning. Tomorrow we recap everything before the group project starts on Day 16.

THE PIPELINE

9 SKILLS

WHAT YOU BUILT

- ✓ PDF upload → PyPDF2 (Day 11)
- ✓ Text cleaning → regex (Day 11)
- ✓ TF-IDF vectors → scikit-learn (Day 13)
- ✓ Cosine similarity → match score (Day 13)
- ✓ Missing keywords → feature analysis (Day 13)
- ✓ Streamlit UI → widgets + layout (Days 4-5)
- ✓ Bar chart → st.bar_chart (Day 7)
- ✓ Data display → st.dataframe (Day 6)
- ✓ Git commit → GitHub (Day 3)

CHECKLIST

5 ITEMS

BEFORE TOMORROW

- ✓ Test your app with a real resume PDF
- ✓ Commit to GitHub (**green square!**)
- ✓ Try matching your OWN resume to a job description
- ✓ Email hello@zlaark.com if something broke
- ✓ Rest — tomorrow is recap, then Day 16 is the group project

DAYS 15-16

3 STAGES

WHAT'S NEXT

-  **Day 15 — Recap & doubt clearing**
Consolidate everything from Days 9-14. Open Q&A. Fix shaky concepts.
-  **Day 16 — Group project begins**
Groups formed, projects picked, sprints start. The resume matcher pattern is in 4 of the 10 project ideas.
-  **Phase 4 — Guided Group Project**
7 days of building. 5 sprints. Demo Day on Day 25.